

Parallel Metric Skip Lists and Nearest Neighbor Search

Xiangyun Ding
University of California,
Riverside
xding047@ucr.edu

Rohin Garg
Massachusetts Institute
of Technology
rohin@mit.edu

Yan Gu
University of California,
Riverside
ygu@cs.ucr.edu

Yihan Sun
University of California,
Riverside
yihans@cs.ucr.edu

Abstract

The metric skip list is a data structure designed for efficient nearest and k -nearest neighbor search in metric spaces. For many real-world datasets with reasonable distributions—specifically, those with a *constant expansion rate*—it supports $\tilde{O}(n)$ construction time and $O(k \log n)$ query time, where n is the input size and k is the number of nearest neighbors in queries. Notably, unlike alternative approaches, it does not require a bounded aspect ratio, making it more flexible for input data distributions. However, the inherently sequential nature of its original construction has, to our knowledge, precluded any existing parallel algorithm.

In this paper, we present highly parallel and work-efficient algorithms for constructing metric skip lists. Under the assumption of a constant expansion rate, our approach achieves an expected work of $O(n \log n)$ and a polylogarithmic span with high probability. Our design is based on novel algorithmic insights that improves the sequential procedure, enabling a divide-and-conquer strategy that facilitates parallelism while maintaining efficiency.

With our algorithms, we can also support improved bounds for relevant applications using nearest neighbor as building blocks, including bichromatic closest pair (BCP), density-based clustering, and k -NN graph construction, among others. To our knowledge, many of these results represent the first solutions to achieve both work efficiency and polylogarithmic span, relying solely on the assumption of a constant expansion rate.

CCS Concepts

• **Theory of computation** → **Parallel algorithms**; **Data structures design and analysis**; **Nearest neighbor algorithms**.

Keywords

Parallel Algorithm, Nearest Neighbor Search, Randomized Algorithm, Metric Space, Metric Skip Lists

1 Introduction

Nearest neighbor search (NNS) is a fundamental algorithmic primitive with extensive applications in computational geometry, computer graphics, machine learning, computer vision, and various other fields within artificial intelligence. In many practical scenarios, the distance between two points is defined using measures other than the standard Euclidean metric and L_p norm. Commonly employed alternatives include cosine similarity (related to angular distance), Jaccard distance [38] (set-based similarity), tree/graph distances, Mahalanobis distance [45], and Earth Mover’s Distance [54].



This work is licensed under a Creative Commons Attribution International 4.0 License. SPAA '26, July 6–10, 2026, London, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2761-0/26/07.
<https://doi.org/10.1145/3816782.3819182>

All these distance measures satisfy specific properties (defined in Sec. 2) and can be generalized to *metric spaces*.

Many efficient data structures [8, 15, 22, 32, 37, 40, 42, 55] have been developed specifically for NNS in metric spaces. Unfortunately, finding (or even approximating) nearest neighbors in general metrics requires linear time [6, 8]. Therefore, most of these data structures rely on assumptions regarding a “good” data distribution to achieve high efficiency. Two widely-studied properties include low expansion rate, which indicates that the density of points in the metric space changes smoothly, and bounded aspect ratio, which indicates that the scale of the dataset (maximum vs. minimum distance) is bounded. We define these two concepts formally in Sec. 2.

Under certain assumptions, the two canonical theoretical data structures for exact NNS in metric spaces are the *metric skip list* introduced by Karger and Ruhl in 2002 [40] and the *cover tree* proposed by Beygelzimer et al. in 2006 [8]. Their best-known bounds are shown in Tab. 1. The cover tree was long regarded as the standard theoretical solution for NNS in metric spaces since 1) its construction and query bounds are better by a factor of $O(\log \log n)$ where n is the input size, and 2) it has a work-efficient parallel version with polylogarithmic span [32]. However, in addition to a low expansion rate, the cover tree also assumes a bounded aspect ratio¹ for both the original cover tree [8] and its parallel version [32]. This assumption can be strong: in L_p metrics, even a simple quad/octree can achieve the same construction and query bounds assuming a bounded aspect ratio [9]. By contrast, metric skip lists only assume a constant expansion rate to achieve their theoretical guarantees.

Therefore, this gap motivates a natural question for NNS: for metric skip lists, can we achieve the same construction time (work) as cover trees, as well as high parallelism (polylogarithmic span), while maintaining their flexibility of solely relying on the basic low expansion rate assumption? This would make metric skip list a theoretically powerful building block for NNS queries, enabling downstream applications to achieve strong bounds under minimal assumptions. However, designing such an efficient and parallel solution remained open. In this paper, we answer this question affirmatively. In particular, we present new algorithms for metric skip lists that achieve the same $O(n \log n)$ work as cover trees (modulo randomization) for construction with polylogarithmic span, as well as the same asymptotic query bounds, under weaker assumptions.

We note that achieving the strong cost bounds, both in work (overcoming the $O(\log \log n)$ overhead) and span (achieving parallelism), is highly non-trivial. Several factors contribute to this difficulty. The first reason is that the original sequential algorithm itself is substantially complicated. The algorithm given in the original paper [40, 55] requires $O(n \log n \log \log n)$ work, which incurs an $O(\log \log n)$ factor overhead as compared to cover trees. While

¹A recent work [23] showed a sequential cover tree variant without this assumption; to our knowledge, no parallel version of this variant or easy fix of [32] is known.

	Original Cover Tree and Its Parallel Version	Metric Skip Lists
Proposed in	Beygelzimer et al. [8]; analysis later corrected by Curtin [19] and Elkin and Kurlin [22]	Karger and Ruhl [40]
Assumptions for the stated bounds	Constant expansion rate, and bounded aspect ratio	Only constant expansion rate
Sequential Construction Time	$O(n \log n)$	$O(n \log n \log \log n)$ <i>whp</i> [40]. Improved to $O(n \log n)$ expected [this paper]
Nearest Neighbor Search Time	$O(\log n)$	$O(\log n \log \log n)$ <i>whp</i> [40]. Improved to $O(\log n)$ expected [this paper]
Parallel Construction Work	$O(n \log n)$ expected [32]	$O(n \log n)$ expected [this paper]
Parallel Construction Span	$O(\log^3 n \log \log n)$ <i>whp</i> [32]	$O(\log^3 n)$ <i>whp</i> [this paper]

Table 1: Comparison between cover trees and metric skip lists for exact NNS in metric spaces. The stated bounds assume a constant expansion rate; the cover-tree bounds additionally use a bounded aspect ratio. For both data structures, the assumptions are needed for the displayed asymptotic bounds, not for correctness: without them, both still find exact nearest neighbors, but the expansion rate and the aspect ratio (for cover trees) appear in the cost bounds.

that paper shared some high-level ideas regarding the possibility of achieving an efficient construction with $O(n \log n)$ work, no algorithmic details or formal analysis were provided. Hence, the first major contribution of this paper is to present an $O(n \log n)$ -work sequential construction algorithm for metric skip lists and its analysis, which are presented in Sec. 3. As mentioned in [40], maintaining $O(n \log n)$ additional pointers can avoid the $O(\log \log n)$ overhead in queries. However, we note that achieving this requires a major redesign of the algorithm, as we show in detail in Sec. 3, since computing each of these pointers can require $O(\log n)$ work in the worst case. To bound the overall work in construction, we analyze and amortize these additional costs against other operations to achieve the improved bounds.

We note that achieving the strong cost bounds, both in work (overcoming the $O(\log \log n)$ overhead) and span (achieving parallelism), is highly non-trivial. Several factors contribute to this difficulty. The first reason is that the original sequential algorithm itself is substantially complicated. The algorithm given in the original paper [40, 55] requires $O(n \log n \log \log n)$ work, which incurs an $O(\log \log n)$ factor overhead as compared to cover trees. While that paper shared some high-level ideas regarding the possibility of achieving an efficient construction with $O(n \log n)$ work, no algorithmic details or formal analysis were provided. Hence, the first major contribution of this paper is to present an $O(n \log n)$ -work sequential construction algorithm for metric skip lists and its analysis, which are presented in Sec. 3. As mentioned in [40], maintaining $O(n \log n)$ additional pointers can avoid the $O(\log \log n)$ overhead in queries. However, we note that achieving this requires a major redesign of the algorithm, as we show in detail in Sec. 3, since computing each of these pointers can require $O(\log n)$ work in the worst case. To bound the overall work in construction, we analyze and amortize these additional costs against other operations to achieve the improved bounds.

The second reason is the complicated computational structure of metric skip lists for both queries and construction. The high-level idea of metric skip list is to first permute all points randomly, obtaining the *priority* for each point (lower rank means higher priority). We denote the permuted point with rank i as s_i . For each point s_i , the goal is to maintain an index to retrieve a constant number α of random (specifically, highest-priority) points within

any given distance r from s_i . Such indices are referred to as **finger lists**. Sequentially, the finger lists are built from the lowest-priority point (s_n) to the highest (s_1), one at a time. Each point s_i maintains a sequence of finger lists: its first finger list simply includes the next α points after s_i ; we denote this finger list as F . Then, we iterate through the rest of the points s_j from $j = i + 1$ to n . If s_j is closer to s_i than the current points in F , we replace the farthest point in F with s_j , store the updated F as the next finger list for s_i , and proceed to the next j . Due to the random permutation, Karger and Ruhl [40] showed that each point has $O(\alpha \log n)$ finger lists *whp*. These finger lists will navigate queries to find the nearest neighbor in a random-walk manner; a query for a point q will start from s_1 , using its finger list to identify a (roughly) closer point s_{next} , and advance to the finger list of s_{next} to continue the process.

By definition, a trivial construction process takes $O(n)$ work per point ($O(n^2)$ total work), assuming α is a constant. The key insight for a more efficient construction algorithm is that, when constructing the finger list for s_i , we can consult the existing finger lists of $s_{i+1} \dots s_n$. At a high level, this process resembles a random walk that queries s_i in the already constructed finger lists of $s_{i+1} \dots s_n$. Along the way, we evaluate the encountered points and build the finger lists for s_i accordingly. This yields $\tilde{O}(n)$ construction work rather than the naïve quadratic cost.

Given this computational structure, work-efficient parallelization of this process is highly non-trivial. The iterative, element-by-element construction of the lists appears inherently sequential: computing the finger lists for s_i may consult, and thus seemingly depend on, the finger lists of the subsequent α points. While the true dependencies are much sparser (computing the finger lists for s_i may require one finger list of s_j ($j > i$) but not all finger lists), analyzing the actual dependence graph remains very difficult due to the complications of both isolating these true dependencies and the randomization inherent in the algorithm.

To overcome this obstacle, our solution is somewhat counterintuitive. We add artificial synchronization barriers to the dependence graph—in a divide-and-conquer manner. For instance, at the top level, we compute the finger lists for points $s_1, \dots, s_{n/2}$ and the finger lists for points $s_{n/2+1}, \dots, s_n$ in parallel. By definition, the second (right) subproblem is fully settled, but the first (left) one requires additional care to handle cross-boundary dependencies.

Note that the random walk starts from the high-priority points and proceeds toward the lower-priority points. Hence, for a point s_i in the left subproblem, the random walk can be blocked without knowing the points on the right. This process must terminate when the random walk wishes to advance to the finger list of a point s_j that has not been fully built. In this case, we say s_j **controls** s_i . By introducing the “artificial barriers” in the divide-and-conquer manner, we prove that the “control” dependences among the input points form a tree, thereby facilitating more intuitive reasoning and analysis. We show that the random-walk property guarantees the tree height as $O(\log n)$ *whp*. Combined with the divide-and-conquer paradigm, we show that the dependence depth for the entire computation is $O(\log^2 n)$ *whp*, and the total span is $O(\log^3 n)$ *whp*. We provide more details of our algorithm and analysis in Sec. 5. Combining all these techniques, we present the main theorem of this paper below.

THEOREM 1.1. *Given a set of n points with a constant expansion rate, the metric skip list can be constructed in $O(n \log n)$ expected work and $O(\log^3 n)$ span *whp*. A nearest neighbor query can be answered in $O(\log n)$ expected time.*

Nearest neighbor search is a fundamental primitive in computational geometry [2, 34], machine learning [12, 13], and data mining [56]. Our results yield the best-known parallel theoretical bounds for many applications, including bichromatic closest pair, density-based clustering, and k -NN graph construction. Due to space limitations, these applications are discussed in the full version of this paper [20].

2 Preliminaries

We use the term $O(f(n))$ **with high probability (whp)** in n to indicate that the bound $O(g(k) \cdot f(n))$ holds with probability at least $1 - 1/n^k$ for any $k \geq 1$ for some function $g(k)$. When the context is clear, we drop “in n ”. Our notation is stronger than the high probability in some other work that only assumes a given k that exists. This is necessary in analyzing the expected work bound for our proposed algorithms.

Computational Model. When analyzing the algorithms, we use the standard work-span model for fork-join parallelism with binary forking [10, 16]. We assume a set of threads that share a common memory. Each thread supports standard RAM instructions, a fork instruction that forks two new child threads, and a constant-time test-and-set atomic operation. When both children complete, the parent thread continues. A computation starts with a single root thread and finishes when that root thread finishes. The **work** of an algorithm is the total number of instructions. The **span** (depth) is the length of the longest sequence of dependent instructions in the computation. The randomized work-stealing scheduler can execute such a computation in $W/P + O(S)$ time *whp* in W [5, 11, 31]. We say a parallel algorithm is **work-efficient** if its work is $O(W)$, where W is the work of the best-known sequential algorithm.

Low-Expansion Metric Space. The efficiency of many nearest-neighbor search algorithms and data structures relies on the assumption of low expansion rate of a metric space. A metric (X, d_X) is defined on a set X with a distance function $d : X \times X \rightarrow \mathbb{R}^*$ that satisfies properties for any $x, y, z \in X$:

- (1) $d_X(x, y) = 0 \Leftrightarrow x = y$,
- (2) $d_X(x, y) = d_X(y, x)$, and
- (3) $d_X(x, y) \leq d_X(x, z) + d_X(z, y)$ (the triangle inequality).

When the context is clear, we drop the subscript X . We use *point* or *record* interchangeably to refer to the objects in the metric space.

Define $B_X(p, r) = \{x \in X \mid d(p, x) \leq r\}$ as the closed ball centered at point p and containing all points in X at a distance of at most r from p . With clear context, we drop X . We say a metric has **(ρ, c) -expansion** [40] *iff* for all $p \in X$ and $r > 0$,

$$|B(p, r)| \geq \rho \implies |B(p, 2r)| \leq c \cdot |B(p, r)|.$$

The parameter c is referred to as the **expansion rate** of the metric space, and we say a metric has a low or constant expansion rate if $c = O(1)$. Usually we assume ρ is $O(\log |X|)$, which guarantees constant expansion for most real-world datasets. Otherwise, the bounds for the find operation for the metric skip list will include an additive term of $O(\rho)$ [55]. Intuitively, low expansion means a smooth distribution of the points, and rules out the case where, as the ball grows, we encounter a few points, then a long distance with no points, then a sudden and massive increase in the number of points. This case is also less likely in most real-world datasets.

Bounded Aspect Ratio. Although not used in metric skip lists, for completeness, we define the aspect ratio, which is assumed to be bounded for cover trees to achieve the same asymptotic bounds as metric skip lists. Aspect ratio is defined as

$$\Delta = \frac{\max\{d(x, y) \mid x, y \in X\}}{\min\{d(x, y) \mid x, y \in X\}}.$$

To achieve $O(\log n)$ work for construction, cover trees in [8] and the parallel version in [30], require the aspect ratio $\Delta \leq n^\kappa$ for some constant $\kappa > 0$ to achieve the theoretical bounds, while metric skip lists do not have such a constraint.

Notation. In this paper, we study a set of elements in the metric space. A common scenario is to consider multi-dimensional points, so we use *points* to refer to the elements for simplicity, but our algorithm works on a general metric space. Based on the nature of the metric skip list algorithm, we assume the input point set S has been randomly shuffled, and use s_i to denote the i -th point. When the context is clear, we may also use the index (also the priority) i to refer to the point s_i . We use $n = |S|$ as the input size.

The notation used in this paper are summarized in Tab. 2. More details about the notations for *finger lists* will be introduced in Sec. 3.2 along with the background of the metric skip list.

3 The Metric Skip Lists

The metric skip list is a randomized data structure for nearest neighbor search, originally proposed by Karger and Ruhl [40]. In the same work, they introduced the concept of the expansion rate of a metric space (as detailed in Sec. 2), which is often referred to as the KR-dimension. This metric has since been extensively used as a standard assumption for analyzing algorithms in metric spaces. Under the assumption of a constant expansion rate, Karger and Ruhl showed that an metric skip list of n points can be constructed with $O(n \log n \log \log n)$ work, while enabling nearest neighbor queries in $O(\log n \log \log n)$ cost.

The main contribution of this paper is to improve the cost bounds

General Terms	
S	$S = \{s_{1..n}\}$: A randomly shuffled sequence of input points.
c	The constant expansion rate.
$d(p, q)$	The distance between points p and q .
The Finger Lists	
α	The sample size, or the size of each finger list.
$\mathcal{F}_i$	An array of finger lists for point i , reversely sorted by radii.
$\mathcal{F}_i[k]$	The k -th finger list in $\mathcal{F}_i$.
$\mathcal{F}_i(r)$	Returns the first finger list in $\mathcal{F}_i$ with radius <i>no more than</i> r . This can be performed in $O(\log \mathcal{F}_i) = O(\log \log n)$ time whp by binary search.
$\text{align}(\mathcal{F}_i[k], r)$	Returns $F^* = \mathcal{F}_i(r)$, by traversing the array $\mathcal{F}_i[\cdot]$ up or down from $\mathcal{F}_i[k]$.
Let $F = \mathcal{F}_i[k]$ be the k -th of the finger lists of point i . It maintains:	
$F.\text{list}$	All points in this finger list. With clear context, we directly use F to represent $F.\text{list}$.
$F.\text{radius}$	The radius of the finger list, i.e., $\max_{j \in F} d(s_i, s_j)$.
$F.\text{advance}$	A mapping from each element in F to another finger list. For each $j \in F$, $F.\text{advance}[j]$ is a pointer to the finger list $\mathcal{F}_j(r)$, where $r = F.\text{radius}$.
General Algorithm Descriptions	
cur	(or s_{cur}) The <i>focus point</i> . The current point processed by the construction/query algorithms, or the random walk scheme.
next	(or s_{next}) The next focus point.
F^*	The <i>focus list</i> . One finger list in $\mathcal{F}_{\text{cur}}$, where we will find the next focus point next from.

Table 2: Notations used in algorithms and analysis.

of metric skip lists. Sequentially, we improve the work bounds for construction and query to $O(n \log n)$ and $O(\log n)$, respectively, which are presented in Sec. 4. More importantly, we provide the first parallel version of the metric skip list, where the construction is work-efficient with polylogarithmic span, given in Sec. 5.

Before presenting our contributions, we first overview the original construction of metric skip lists in this section. We start with the underlying intuition, followed by the formal definition and algorithms for metric skip lists. This background facilitates a better understanding of our improved algorithms and their analysis.

3.1 The Random-Walk Procedure

The metric skip list is designed for nearest neighbor search, motivated by the intuition of a random-walk procedure to reach the nearest neighbor of a query point q among an input set S using *sampling*. At a high level, one starts from an arbitrary point $p \in S$ and iteratively attempts to walk to a closer point. By the triangle inequality, if a point closer to q than p exists, it must reside within the ball $B_S(p, 2d(p, q))$. Therefore, the *sampling scheme* inspects a random sample of α' points in $B_S(p, 2d(p, q))$, moves to the one closest to q as the next candidate, and repeats this process. This procedure is outlined in Alg. 1. With an appropriately chosen sample rate, each step has a sufficient probability of “making progress” toward a significantly closer point, despite a small probability of moving to a farther one. Existing work showed that the following

Algorithm 1: The random-walk process to q 's nearest neighbor based on sampling [40, 55]

Input: A set of points S and a query point q

```

1 Let  $p$  be an arbitrary point in  $S$ 
2 repeat
3   Let  $S'$  be a random sample of  $2c^3$  points in  $B_S(p, 2d(p, q))$ 
4   Update  $p$  as the closest point to  $q$  in  $S'$ 
5 until  $p$  is the nearest neighbor of  $q$  in  $S$ 

```

lemma holds by setting $\alpha' = 2c^3$ for an expansion rate c .

LEMMA 3.1 ([40, 55]). *Given a metric space (S, d_S) with expansion rate c , the random walk in Alg. 1 reaches the nearest neighbor of q in $O(\log |S|)$ iterations whp.*

While a formal proof is provided in [40, 55], we include a proof sketch here to facilitate the understanding of our improved algorithm and its analysis. To analyze this random-walk procedure, we define the **rank** of a point $p \in S$ with respect to the query point q as its position in the sorted order of distances to q . Specifically, p has rank k if it is the k -th closest point to q in S . We will show that throughout the random walk, the rank of p decreases rapidly, dropping to 1 in $O(\log n)$ iterations whp.

Let the current point being processed be p with rank k . We denote $\delta = d(p, q)$ and let B represent the ball $B_S(p, 2\delta)$. We first prove $|B| \leq c^2 k$. Since p has rank k , there are at most k points in $B_S(q, \delta)$. For any point $p' \in B$, the triangle inequality implies $d(q, p') \leq d(q, p) + d(p, p') \leq \delta + 2\delta < 4\delta$. Thus, all points in B are contained in $B_S(q, 4\delta)$. By the definition of the expansion rate and the fact that $|B_S(q, \delta)| \leq k$, we obtain $|B| \leq |B_S(q, 4\delta)| \leq c^2 k$.

Therefore, the next point is selected by sampling $\alpha' = 2c^3$ points from at most $c^2 k$ candidates. We show that, in this case, the closest point is likely to have a rank smaller than $k/2$. The probability that all $2c^3$ samples have rank at least $k/2$ is at most $\left(1 - \frac{k/2}{c^2 k}\right)^{2c^3} = \left(1 - \frac{1}{2c^2}\right)^{2c^3} \leq e^{-c}$. Thus, the rank of the next candidate point is halved with probability at least $1 - e^{-c}$. In the complementary case, the candidate point remains in B , which has rank at most $c^2 k$. This implies that even in the worst case, the rank increases by a factor of at most c^2 with low probability e^{-c} . Combining the two cases, one can verify that the rank decreases by a constant factor in expectation for any $c \geq 1$, ensuring that the nearest neighbor is reached in $O(\log n)$ iterations whp.

3.2 The Structure of Metric Skip Lists

With the background, we now introduce the metric skip lists. The metric skip lists are built upon the sampling idea introduced in Alg. 1. At a high level, we aim to build a structure such that for each point $s_i \in S$ and a given radius r , the data structure can easily look up $\mathcal{F}_i(r)$ as a random sample of α points in ball $B_S(s_i, r)$. While r can be an arbitrary real number, for each point, there are at most n discrete values of r that are meaningful. Therefore, the idea is to randomly shuffle all the points, and for any of the meaningful values of r , we store the α points with the highest priorities.

Definition of Metric Skip Lists and Finger Lists. More specifically, a metric skip list $\mathcal{F}$ maintains a structure $\mathcal{F}_i$ for the i -th point s_i , which is a series of **finger lists**. Each *finger list* contains α points

Example: finger lists of point ① ($\mathcal{F}_1$). Assume $\alpha = 2$.

Priority	②	③	④	⑤	⑥	⑦
Distance to ①	6	5	8	2	7	4

$\mathcal{F}_1$ includes five finger lists:

$\mathcal{F}_1[1]$	radius = 6	list = $\langle ③, ② \rangle$	advance = $[③ \mapsto \mathcal{F}_3(6), ② \mapsto \mathcal{F}_2(6)]$
↓ ⇒ [add ④ (distance>6), fail] ⇒ [add ⑤ (distance<6), success, evict ②]			
$\mathcal{F}_1[2]$	radius = 5	list = $\langle ⑤, ③ \rangle$	advance = $[⑤ \mapsto \mathcal{F}_5(5), ③ \mapsto \mathcal{F}_3(5)]$
↓ ⇒ [add ⑥ (distance>5), fail] ⇒ [add ⑦ (distance<5), success, evict ③]			
$\mathcal{F}_1[3]$	radius = 4	list = $\langle ⑤, ⑦ \rangle$	advance = $[⑤ \mapsto \mathcal{F}_5(4), ⑦ \mapsto \mathcal{F}_7(4)]$
↓ no more points. Add the tail by removing the farthest points one by one			
$\mathcal{F}_1[4]$	radius = 2	list = $\langle ⑤ \rangle$	advance = $[⑤ \mapsto \mathcal{F}_5(2)]$
$\mathcal{F}_1[5]$	radius = 0	list = $\emptyset$	advance = $\emptyset$

Tail

Figure 1: An example of the finger lists of Point 1. The blue fields are used in the original paper [40, 55]. The orange fields are auxiliary metadata maintained in this paper. The definition of the fields are introduced in Tab. 2.

with its corresponding *radius*, which is the largest distance among all points in this finger list to s_i . The first finger list $\mathcal{F}_i[1]$ simply consists of the next α points, i.e., $\{s_{i+1}, \dots, s_{i+\alpha}\}$. We then iterate through the rest of the points in order, and if a point is closer than the radius of the current list, we will replace the farthest point with the new point, create a new finger list, and make it the next finger list in $\mathcal{F}_i$. In this paper, we call a point j an **evictor** of a finger list $\mathcal{F}_i[k]$, if j is the point that successfully evicts the farthest point in $\mathcal{F}_i[k]$, thereby forming $\mathcal{F}_i[k+1]$. An example is shown in Fig. 1.

After we iterate through all points, we continue the process of removing the farthest point in the finger list, and construct α more finger lists at the end. We call these finger lists the **tail** in $\mathcal{F}_i$. This concept is useful for the correctness of the algorithms.

Properties of a Finger List. Although the definition may suggest $O(n)$ worst-case size for each finger list, due to the random permutation, the actual size is much smaller *whp*. Existing work has shown the following lemma about the size of finger lists.

LEMMA 3.2. *Given a randomly permuted point set $S = \{s_{1..n}\}$ and a constant α , $|\mathcal{F}_i| = O(\alpha \log n)$ whp.*

It is easy to see that this lemma is true based on the random permutation. In fact, $|\mathcal{F}_i| \leq \alpha \cdot H_n$ where H_n is the harmonic series.

Note that for any point i , all finger lists in $\mathcal{F}_i$ are naturally sorted by distance (high to low). Therefore, looking for $\mathcal{F}_i(r)$ can be easily performed by a binary search in $O(\log \log n)$ time. For each i and r , $\mathcal{F}_i(r)$ contains the α points with highest priority in radius r , forming a random sample set. In this paper, we set $\alpha = 16c^3$. This is different from the parameter in Alg. 1, and we will explain the reason later when we present the construction and query algorithms in Sec. 3.3. We also note that we use a different α from the original paper [40, 55] to simplify the analysis, but in both cases α is a function of c and is a constant with constant expansion rate.

The Organization and Notation of Finger Lists. In the original paper, each finger list only maintains the corresponding list of α points and its radius. Therefore, we represent each finger list F as a structure described in Tab. 2. In particular, we use $F.list$ to store the list of points. With clear context, we directly use F as $F.list$. We use $F.radius$ to represent the radius of F .

In our paper, we will further augment each list with useful pointers (the *.advance* pointers) to accelerate the algorithms. We introduce them in detail in Sec. 4 and 5. The major overhead of the original algorithm stems from using binary search to look up $\mathcal{F}_i(r)$,

Algorithm 2: Construct($s_{1..n}$) in [40]

Input: A list of randomly permuted points $s_{1..n}$, and α .

Output: The metric skip lists $\mathcal{F}_{1..n}$.

```

1 for  $i \leftarrow n$  downto 1 do
2   | Build-Finger( $i$ )           // building the finger lists for the  $i$ -th point
3 Function Build-Finger( $i$ )
4    $\mathcal{F}_i[1].list \leftarrow \{s_{i+1}, s_{i+2}, \dots, s_{\min\{i+\alpha, n\}}\}$ 
5    $cur \leftarrow i + \alpha$ 
6   while true do
7      $F \leftarrow$  the last finger list of  $s_i$  built so far
8      $r \leftarrow \max\{d(s_{cur}, s_i) + F.radius, 2d(s_{cur}, s_i)\}$ 
9      $F^* \leftarrow \mathcal{F}_{cur}(r)$ 
10    if  $\exists s_j \in F^*$  s.t.  $d(s_j, s_i) < \max\{F.radius, d(s_{cur}, s_i)\}$  then
11      |  $nxt \leftarrow$  the first  $j$  with this property in  $F^*$ 
12      | if  $d(s_{nxt}, s_i) < F.radius$  then
13        |  $F_{new} \leftarrow$  replacing the farthest point in  $F$  with  $s_{nxt}$ 
14        | Append  $F_{new}$  to  $\mathcal{F}_i$ 
15    else
16      | if  $|F^*| < \alpha$  then Build the tail and break
17      | else  $nxt \leftarrow$  the last index in  $F^*$ 
18     $cur \leftarrow nxt$ 

```

which costs $O(\log \log n)$ *whp* each time. Our augmented pointers are mainly used to avoid this cost, which improves the bound even for the existing sequential algorithm.

Although the definition of metric skip list may suggest $O(n^2)$ work in the worst case for construction, since the total size of all finger lists is $O(\alpha n \log n)$ *whp*, one can actually use a more efficient algorithm to build them. Next, we present the original construction and query algorithms proposed in Karger and Ruhl [40] (referred to as the KR algorithm), where construction takes $O(n \log n \log \log n)$ work and a nearest neighbor query takes $O(\log n \log \log n)$ work.

3.3 The KR Algorithms

In this section, we review the original algorithm from [40] for the metric skip lists introduced above. With the metric skip lists, we can efficiently query the nearest neighbor, k -nearest neighbor, or range query (count or list the number of points within a given radius). All query algorithms are similar, and thus we use nearest neighbor query as an example.

Note that the KR algorithms rely on the function $\mathcal{F}_i(r)$ that finds the last finger list in $\mathcal{F}_i$ with radius no more than r , as introduced above. This is achieved by binary searching in $\mathcal{F}_i$ and has $O(\log \log n)$ cost *whp* due to Lemma 3.2. We will later show how to avoid using $\mathcal{F}_i(r)$ in our algorithm in Sec. 4.

The Construction Algorithm. The KR construction algorithm builds the metric skip list from $\mathcal{F}_n$ down to $\mathcal{F}_1$, in descending order of indexes. We present the algorithm in Alg. 2. Each $\mathcal{F}_i$ will leverage finger lists for those with indexes greater than i , which are already available. By doing so, the building $\mathcal{F}_i$ avoids an exhaustive search over all subsequent points. This will be achieved by searching s_i in existing finger lists—following the random walk scheme in Sec. 3.1—and on the way, building all the finger lists in $\mathcal{F}_i[\cdot]$.

We now explain Alg. 2 in detail. We start with the first finger list, which by definition contains the α points right after s_i (line 4). Let this finger list be F . Throughout the algorithm, F represents the last built finger list, and the goal is to build the next F . The

goal is then to find the *evictor* of F , i.e., the first point after $s_{i+\alpha}$ with a smaller distance, within the random walk. As mentioned, the algorithm simulates a random walk to query s_i . We maintain a variable cur in Alg. 2 to represent the current point in the random walk. Note that $cur > i$ must be later than i and have its finger lists ready. We call cur the **focus point** at this stage and say we **focus on** cur . During the process, we will attempt to look for the evictor in $\mathcal{F}_{cur}$, and find the next focus point (the nxt variable in Alg. 2) for the random walk. We start with $cur = i + \alpha$ (line 5).

Based on the random walk scheme, we need to focus on a ball B^* around s_{cur} with radius $r \geq 2d(s_{cur}, s_i)$ for quick convergence. Moreover, if an evictor j exists, it must satisfy $d(s_j, s_i) < F.radius$, and thus we want the radius of B^* to also cover $r \geq F.radius + d(s_i, s_{cur})$ to avoid missing any evictor. Therefore, the radius r is calculated in line 8. By definition of finger lists, this ball B^* corresponds to $\mathcal{F}_{cur}(r)$, which contains α random points within B^* . It can be easily located by a binary search on $\mathcal{F}_{cur}[\cdot]$. Let $F^* = \mathcal{F}_{cur}(r)$. Similarly, we call F^* the **focus list** at this stage of the algorithm, and say that we **focus on** $\mathcal{F}_{cur}(r)$.

Among all points in the focus list, if any point is closer to s_i than s_{cur} or than the radius of the last built finger list, we will set the one with the smallest index (highest priority) as the next focus point nxt . If nxt happens to be also within $F.radius$, it is exactly the evictor. Hence, we can construct the next finger list accordingly and append it to $\mathcal{F}_i$ (lines 13–14).

If all points turn out to be farther (line 16), we first check whether F^* is a tail list (i.e., contains fewer than α points). If so, we know we have exhausted the points in $B(s_{cur}, F.radius)$, and the random walk terminates. Therefore, we build the tail and exit (line 16). If F^* is a regular finger but no closer point is found, we set nxt to the last index in F^* to continue this process (line 17). This indicates that the random walk temporarily goes to a farther point than s_{cur} .

The construction algorithm is illustrated in Fig. 2, which is presented with the parallel algorithm. The first four subfigures (a)–(d) are equivalent to the construction process of point $i = 1$ based on the already-constructed finger lists for points 2 to 7. At a high level, the construction of the finger lists $\mathcal{F}_i$ for a point s_i resembles the random-walk process in Alg. 1. The series of focus points are the points visited in the random walk, and will “advance” gradually to the right (larger indexes/lower priorities) until we reach the end of the metric skip list. This is performed by maintaining the variables cur, F^*, nxt in the algorithm. cur denotes the current focus point. F^* is the focus list, which is a finger list of cur . Finally, nxt is selected from F^* as the next focus point. The focus point s_{cur} in each iteration can get closer to s_i (line 11) or farther (line 17), but will finally visit all “useful” points that should appear in $\mathcal{F}_i$.

However, differently from Alg. 1, the samples in Alg. 2 are now decided by the priorities of a predefined random permutation. Hence, the algorithm is also changed to jump to the first closer point (line 11) rather than the closest point. If no such points are found, we take the last point in F^* (line 17) to continue advancing the focus points. As such, we do not rely on any randomness of the points in the future iterations, so the probability distributions of the random walk in different iterations remain independent. Due to this change, we need a larger probability to find a closer point, so α is increased to $16\epsilon^3$ (or larger) to offset this difference [40]. This leads to the following result.

Algorithm 3: *NearestNeighbor*($q, S, \mathcal{F}$) from [40]

Input: A list of randomly permuted points $S = s_{1..n}$, the query point q , and the metric skip list $\mathcal{F}$.

Output: The closest point to q in S .

```

1  $cur \leftarrow 1, m \leftarrow s_1$ 
2 while true do
3    $r \leftarrow 2d(s_{cur}, q)$ 
4    $F^* \leftarrow \mathcal{F}_{cur}(r)$ 
5   if  $\exists s_j \in F^*$  such that  $d(s_j, q) < d(s_{cur}, q)$  then
6      $nxt \leftarrow$  the first  $j$  with this property in  $F^*$ 
7     if  $d(s_{nxt}, q) < d(m, q)$  then  $m \leftarrow s_{nxt}$ 
8   else
9     if  $|F^*| < \alpha$  then break
10    else  $nxt \leftarrow$  the last index in  $F^*$ 
11   $cur \leftarrow nxt$ 
12 return  $m$ 
```

THEOREM 3.1 ([40]). *Given a randomly permuted point set $S = \{s_{1..n}\}$ with a constant expansion rate and a constant α , the metric skip list can be constructed in $O(n \log n \log \log n)$ work whp.*

The Query Algorithms. The nearest neighbor query algorithm in metric skip lists is very similar to the construction algorithm (see Alg. 3). The process resembles a simplified version of the construction of the finger lists of a point, with the difference that we only need to keep track of one point instead of α points. We maintain the closest point that has been seen so far (line 7) and output it.

We can also adapt Alg. 3 for other queries. For k -nearest neighbor search, we will keep a set of k points, which is very similar to the construction algorithm that maintains α points. For range query with distance δ , we can set the search radius $r \leftarrow d(s_i, q) + \max(\delta, d(s_i, q))$ on line 3.

THEOREM 3.2 ([40]). *Given a metric skip list maintaining points with a constant expansion rate, nearest neighbor and k -nearest neighbor can be computed in $O(\log n \log \log n)$ and $O(k \log n \log \log n)$ work whp, respectively; range query can be answered in $O((k' + \log n) \log \log n)$ work whp where k' is output size.*

4 A More Efficient Sequential Algorithm

In this section, we will explain our improved sequential algorithms for nearest-neighbor queries and the construction of the metric skip lists. As mentioned, the main overhead of the original algorithm comes from looking up $\mathcal{F}_i(r)$ using binary search. By replacing $\mathcal{F}_i(r)$ with two algorithmic primitives: an algorithm $align(\mathcal{F}_i[k], r)$ and an array of augmented pointers $\mathcal{F}_i[k].advance[j]$ for each finger list, we can achieve $O(n \log n)$ expected work for construction and $O(\log n)$ expected work for nearest-neighbor query, under the same assumptions. We describe the definitions of *align* and *advance* in Tab. 2, and introduce more details below.

Since our algorithms are considerably more complicated than the original ones, we will start with the query algorithm in Sec. 4.1 and its analysis. We will then introduce the construction algorithm in Sec. 4.2 as well as an amortized analysis for its cost bounds.

4.1 The Query Algorithms

The key idea in our query algorithm, as shown in Alg. 4, is to use the *align* algorithm and the *advance* pointers to accelerate the

Algorithm 4: *NearestNeighbor*($q, S, \mathcal{F}$) in $O(\log n)$ cost**Input:** A list of randomly permuted points $S = s_{1..n}$, the query point q , and the metric skip list $\mathcal{F}$.**Output:** The closest point to q in S .

```

1  $cur \leftarrow 1, k \leftarrow 1$ 
2  $m \leftarrow s_1$  // the closest point to  $q$  found so far
3 while true do
4    $r \leftarrow 2d(s_{cur}, q)$ 
5    $F^* \leftarrow align(\mathcal{F}_{cur}[k], r)$  // aligns to the first list with radius  $\leq r$ 
6   if  $\exists s_j \in F^*$  such that  $d(s_j, q) < d(s_{cur}, q)$  then
7      $nxt \leftarrow$  the first  $j$  with this property in  $F^*$ 
8     if  $d(q, s_{nxt}) < d(q, m)$  then  $m \leftarrow s_{nxt}$ 
9   else
10    if  $|F^*| < \alpha$  then break
11    else  $nxt \leftarrow$  the last index in  $F^*$ 
12   $k \leftarrow F^*.advance[nxt], cur \leftarrow nxt$ 

// This function re-aligns the current finger list to the first list with radius
//  $r' \leq r$  by traversing up or down in  $\mathcal{F}_i[\cdot]$ 
13 Function  $align(\mathcal{F}_i[k], r)$ 
14    $k' \leftarrow k$ 
15   // Based on whether  $r$  is larger or smaller than  $\mathcal{F}_i[k].radius$ , only one
16   // of the two while-loops below will be executed.
17   while  $k' > 1$  and  $\mathcal{F}_i[k'].radius \leq r$  do  $k' \leftarrow k' - 1$  // up
18   while  $\mathcal{F}_i[k'].radius > r$  do  $k' \leftarrow k' + 1$  // down
19   return  $\mathcal{F}_i[k']$ 

```

query process. $align(\mathcal{F}_{cur}[k], r)$ is used to calibrate the index k to k' in $\mathcal{F}_{cur}[\cdot]$, such that $\mathcal{F}_{cur}[k']$ is $\mathcal{F}_{cur}(r)$. This can be implemented easily by traversing $\mathcal{F}_{cur}[\cdot]$ up or down from index k , since all finger lists are sorted by their radii. The process is shown on line 13–16. We will show that whenever $align(\mathcal{F}_{cur}[k], r)$ is called, the destination is likely not too far from k , which bounds the total cost caused by $align$ during the query algorithm (Lemma 4.3). Meanwhile, here we augment each finger list $\mathcal{F}_i[k]$ with an array $advance[j]$, which maps each $j \in \mathcal{F}_i[k]$ to $\mathcal{F}_j(r)$, where $r = \mathcal{F}_i[k].radius$ is the radius of the current list. This allows us to advance to the next focus list quickly without a binary search—when we shift the focus from cur to nxt , we first advance to the finger list in nxt with the same radius using the $.advance$ pointer. We then calibrate around this list by moving up or down to determine the actual focus list. We will later show how the $.advance$ pointers are built efficiently in construction in Sec. 4.2. Here we assume it is correctly computed, so we can introduce our query algorithm. Due to space limit, we only provide proof sketches here and defer the full proofs in the full version [20].

LEMMA 4.1. *Given $\mathcal{F}_i$, there can be $O(1)$ expected lists between $\mathcal{F}_i(r)$ and $\mathcal{F}_i(2r)$.*

Proof sketch. Due to the low expansion rate, $|B(s_i, 2r)|/|B(s_i, r)| \leq c$. Consider the points in $B(s_i, 2r) \setminus B(s_i, r)$ in their random permutation order. A point can only be added to $\mathcal{F}_i$ (thus created a new finger list) before we have seen α points from $B(s_i, r)$ in this order. Hence, the expected number of such points is $\alpha \cdot |B(s_i, 2r)|/|B(s_i, r)| = O(\alpha c) = O(1)$. $\square$

LEMMA 4.2. *The total number of iterations executed on line 15 is $O(\log n)$ in expectation.*

The proof of this lemma is based on Lemma 4.1 and is provided in the full version [20] due to the space limit.

LEMMA 4.3. *The total work in align is $O(\log n)$ in expectation.*

Proof Sketch. Lemma 4.2 has bounded the expected work of the upward moves ($k \leftarrow k - 1$, line 15). Here we only need to bound the cost of downward moves ($k \leftarrow k + 1$, line 16): in total the pointers are moved for $O(\log n)$ times in expectation. This can be achieved by defining a potential function of the ball size (or the rank) and analyzing the random walk of the potential function. We use $\Phi = \log_2 |B(s_{cur}, 2d(s_{cur}, q))|$. As discussed in Sec. 3.3 and Lemma 3.1, after $O(1)$ new elements in the list, the ball size is expected to halve; equivalently, every $O(1)$ downward *align* move decreases Φ by $\Theta(1)$ in expectation. Hence, the number of downward moves is proportional (in expectation) to the total decrease of Φ . Note that Φ can grow in the upward moves. Each time Line 15 is executed, the search radius at most doubles, so Φ can grow by at most a factor of $c = O(1)$. Hence, we need the same expected asymptotic number of downward moves to account for the decrease of Φ . Putting all pieces together gives $O(\log n)$ expected number of downward moves, and the same cost for the *align* function. $\square$

THEOREM 4.1. *Given a metric skip list maintaining points with a constant expansion rate, Alg. 4 computes the nearest neighbor in $O(\log n)$ expected work.*

Proof. Alg. 4 follows the same logic as Alg. 3. The difference is that $\mathcal{F}_{cur}(r)$ is now obtained by using the *advance* pointers and local adjustment by the *align* function, instead of binary searches in $O(\log \log n)$ cost. Lemma 4.3 proves that the total cost in *align* is $O(\log n)$ in expectation. The *advance* pointers are chased once in each iteration in the random walk, so the total cost here is $O(\log n)$ expected. Combining these costs proves the theorem. $\square$

We can change the candidate set from one point to k points, which yields $O(k \log n)$ expected work for k -nearest neighbor query.

4.2 The Sequential Construction Algorithm

With the insights from the query algorithm, we now explain the construction algorithm that will generate the *advance* pointers efficiently. We provide our algorithm in Alg. 5.

The overall idea in Alg. 5 is quite similar to Alg. 4, except that we will now keep α points rather than one point. Hence, in each iteration of the random walk, we first compute the current search radius r , then we use $align(\mathcal{F}_{cur}[k], r)$ to find the finger list $\mathcal{F}_{cur}[k']$ (effectively $\mathcal{F}_{cur}(r)$ in Alg. 2). Then we perform the same check conditions, find the next focus point nxt , and update $\mathcal{F}_i$ if needed. At the end of the iteration, we take the $F^*.advance[nxt]$ pointer to the next finger list we focus on, just like in Alg. 4. As the construction algorithm, we additionally need to build the *advance* pointers: on lines 5–7 and 18–19.

Recall that the definition is $\mathcal{F}_i(r).advance[j] = \mathcal{F}_j(r)$. In the initial finger list $\mathcal{F}_i[1]$, we first set $\mathcal{F}_i[1].advance[j] = \mathcal{F}_j[1]$ for all $j \in \mathcal{F}_i[1]$. However, this finger list can have a higher radius than $\mathcal{F}_i[1]$, so we may need to decrease the radius of $\mathcal{F}_i[1].advance[j]$ until it is at most $\mathcal{F}_i[1].radius$. Here we call a function *Align-All*(F), which checks all j in F and decreases the radius of $F.advance[j]$ until it is at most $F.radius$.

When s_i gets a new finger list F_{new} , we build the $F_{new}.advance[\cdot]$ pointers for the points in it. Note that F_{new} is created by replac-

Algorithm 5: Sequential $O(n \log n)$ Construction Algorithm

```

1 for  $i \leftarrow n$  to 1 do
2    $\text{Build-Finger}(i)$ 
3 Function  $\text{Build-Finger}(i)$ 
4    $\mathcal{F}_i[1].\text{list} \leftarrow \{s_{i+1}, s_{i+2}, \dots, s_{\min\{i+\alpha, n\}}\}$ 
5   foreach point  $s_j \in \mathcal{F}_i[1]$  do
6      $\mathcal{F}_i[1].\text{advance}[j] \leftarrow \mathcal{F}_j[1]$ 
7    $\text{Align-All}(\mathcal{F}_i[1])$ 
8    $\text{cur} \leftarrow i + \alpha$ 
9    $k \leftarrow 1$  // Start from the largest finger list of  $s_{\text{cur}}$ 
10  while true do
11     $F \leftarrow$  the last finger list of  $s_i$ 
12     $r \leftarrow d(s_{\text{cur}}, s_i) + \max(F.\text{radius}, d(s_{\text{cur}}, s_i))$ 
13     $F^* \leftarrow \text{align}(\mathcal{F}_{\text{cur}}[k], r)$ 
14    if  $\exists s_j \in F^*$  such that  $d(s_j, s_i) < \max(F.\text{radius}, d(s_{\text{cur}}, s_i))$ 
15      then
16         $\text{nxt} \leftarrow$  the first  $j$  with this property in  $F^*$ 
17        if  $d(s_{\text{nxt}}, s_i) < F.\text{radius}$  then
18           $F_{\text{new}} \leftarrow$  replacing the farthest point in  $F$  with  $s_{\text{nxt}}$ 
19           $F_{\text{new}}.\text{advance}[\text{nxt}] \leftarrow F^*.\text{advance}[\text{nxt}]$ 
20           $\text{Align-All}(F_{\text{new}})$ 
21          Append  $F_{\text{new}}$  to  $\mathcal{F}_i$ 
22        else
23          if  $|F^*| < \alpha$  then break
24          else  $\text{nxt} \leftarrow$  the last index in  $F^*$ 
25         $k \leftarrow F^*.\text{advance}[\text{nxt}], \text{cur} \leftarrow \text{nxt}$ 
26  Function  $\text{Align-All}(F)$ 
27  foreach point  $s_j \in F$  do
28     $\text{align}(F.\text{advance}[j], F.\text{radius})$ 

```

ing the farthest point in the focus list F with s_{nxt} . For all other points s_j in F_{new} , their *advance* pointers remain unchanged, i.e. $F_{\text{new}}.\text{advance}[j] = F.\text{advance}[j]$. Then, for the evictor s_{nxt} , we set $F_{\text{new}}.\text{advance}[\text{nxt}]$ to $F^*.\text{advance}[\text{nxt}]$. Then we call $\text{Align-All}(F_{\text{new}})$ to align the radius of all $F_{\text{new}}.\text{advance}[\cdot]$ pointers if needed.

THEOREM 4.2. *Given a randomly permuted point set $S = \{s_{1..n}\}$ with a constant expansion rate and a constant α , Alg. 5 builds the metric skip list in $O(n \log n)$ expected work.*

We first show a useful lemma.

LEMMA 4.4. *The radius of F is expected to halve after adding a constant number of new elements.*

Proof. The proof of Alg 5.18 in [55] shows that, $B(i, \mathcal{F}_j + 2\alpha.\text{radius})$ is expected to halve compared to $B(i, \mathcal{F}_j.\text{radius})$. Hence, due to the expansion rate, the radius of F is expected to halve after seeing $2\alpha \lceil \log c \rceil$ new evictors (repeated $\lceil \log c \rceil$ times). $\square$

Proof of Thm. 4.2. We show that the work for Build-Finger is $O(\log n)$ in expectation in each invocation, which implies the theorem. The analysis of Alg. 2 shows that the random walk has length $O(\log n)$ whp, and the proof of Alg. 4 shows that the align has $O(\log n)$ expected work. Hence, the only leftover part is the work spent on building the *advance* pointers.

Recall that the *advance* pointer is defined as $\mathcal{F}_i(r).\text{advance}[j] = \mathcal{F}_j(r)$. We now show that for each s_j , the cost in building all $\mathcal{F}_i[\cdot].\text{advance}[j]$ pointers can be charged to other operations in $\text{Build-Finger}(i)$ asymptotically. Hence, the total work is $O(\log n)$

expected per random walk, and $O(n \log n)$ expected in total.

For each s_{nxt} added to some finger lists of s_i , maintaining its *advance* pointers appears in two phases.

- Phase 1: the first appearance. When s_{nxt} is first added to some finger list F_{new} , we set $F_{\text{new}}.\text{advance}[\text{nxt}] = F^*.\text{advance}[\text{nxt}]$ (line 18). Then we align the radius of $F_{\text{new}}.\text{advance}[\text{nxt}]$ until its radius is at most $F_{\text{new}}.\text{radius}$.
- Phase 2: the follow-up copies. When s_i 's finger lists get updated by F_{new} , for all points s_j not evicted, we copy $F_{\text{new}}.\text{advance}[j] = F.\text{advance}[j]$. Then we keep decreasing the radius of $F_{\text{new}}.\text{advance}[j]$ until its radius is at most $F_{\text{new}}.\text{radius}$.

Phase 1. We first show that the running time of Phase 1 can be charged to the running time of line 13 in the next iteration of the while loop. For $F_{\text{new}}.\text{advance}[\text{nxt}]$, it is initialized as $F^*.\text{advance}[\text{nxt}]$, then it is aligned to have radius at most $F_{\text{new}}.\text{radius}$. For its counterpart, in line 24, F^* is also updated to $F^*.\text{advance}[\text{nxt}]$, then in the next iteration, it is adjusted to have radius at most $r = d(s_{\text{nxt}}, s_i) + \max(F_{\text{new}}.\text{radius}, d(s_{\text{nxt}}, s_i)) = d(s_{\text{nxt}}, s_i) + F_{\text{new}}.\text{radius}$. This is because s_{nxt} will become the s_{cur} , and F_{new} will become the F in the next iteration, and we also have $d(s_{\text{nxt}}, s_i) < F_{\text{new}}.\text{radius}$ because s_{nxt} is inside F_{new} .

Note that $d(s_{\text{nxt}}, s_i) + F_{\text{new}}.\text{radius} < 2F_{\text{new}}.\text{radius}$. Based on Lemma 4.4, in addition to the running time of Line 13, we only need to perform constant expected number of decreases on $F_{\text{new}}.\text{advance}[\text{nxt}]$ to make its radius at most $F_{\text{new}}.\text{radius}$.

Phase 2. A point s_j may show up in several finger lists in $\mathcal{F}_i$, and we need to update $\mathcal{F}_i[\cdot].\text{advance}[j]$ in $\mathcal{F}_j$ by moving down the corresponding *advance* pointers. Here we can charge this cost to copying the finger lists, since we need to pay $O(1)$ cost every time we duplicate s_j in finger lists. Since s_j is in $\mathcal{F}_i[\cdot]$, the radii of these finger lists must be at least $d(s_i, s_j)$. Due to the constant expansion rate, the numbers of points centered at j and i can only differ by a constant factor of c . Hence, the expected number of pointer moving is asymptotically the same as the number of appearances of s_j in $\mathcal{F}_i$, using the same argument as in the proof of Lemma 4.4.

Combining the two cases proves the theorem. $\square$

5 Parallel Construction of the Metric Skip Lists

In Sec. 4, we presented an improved algorithm that constructs the metric skip list in $O(n \log n)$ expected work, and answers queries in $O(\log n)$ expected work. In this section, we further present a parallel algorithm for the construction process. We note that this parallelization is inherently challenging due to the sequential nature of the process, where building each finger list relies on later finger lists that have already been constructed. More specifically, in both our algorithm and the original KR algorithm [40], building each finger list requires to shift the focus to simulate the random walk process. At the stage where the algorithm is focusing on cur , to find the next focus point nxt from the focus list F^* , cur not only relies on (or needs to wait for) nxt , but also other points in F^* . This is because nxt is chosen as the first or last index in F^* satisfying certain conditions (lines 11,17 in Alg. 2), and thus we may need to know the full list to make the decision. This weaves an intricate dependence structure, which itself is also dictated by randomness.

To overcome this challenge and parallelize the algorithm, our

solution is somewhat counterintuitive. We add artificial synchronization barriers to the dependence graph—in a divide-and-conquer manner. We will show that the dependences crossing the synchronization barriers actually form a tree structure, and it is possible to bound the tree depth in $O(\log n)$ *whp*, which enables a polylogarithmic span for the entire algorithm. However, given the complication of even the sequential algorithms, we will present our parallel algorithms in three steps to improve readability. First, in Sec. 5.1, we describe a simple divide-and-conquer algorithm for the construction of metric skip lists. This algorithm is not work-efficient, but it provides useful intuition to achieve parallelism. This algorithm can achieve polylogarithmic span with an $O(\log n \log \log n)$ factor of overhead in work, or slightly better work with $O(n^\epsilon)$ span. Next, we will show the high-level idea on parallelizing the construction algorithm by strictly following the random-walk procedure without doing extra work. This algorithm is given in Sec. 5.2. The key challenge and algorithmic insight is analyzing the dependence structure, which we refer to as the **control tree**. In Sec. 5.3 we prove the $O(\log n)$ height bound of the control tree, then we show how to construct and use it in Sec. 5.4. Finally, we will show how to achieve the work-efficiency by efficiently maintaining the *advance* pointers for each finger list in each step in Sec. 5.5.

5.1 A Simple Work-Inefficient Solution

We start with a simple work-inefficient divide-and-conquer parallel construction algorithm. The idea is to break the input sequence into two equal-size chunks, $s_{1..n/2}$ and $s_{(n/2+1)..n}$, and build their metric skip lists independently. When they both finish, note that all finger lists on the right half are fully ready. However, the points in the left half can only see the points in $s_{1..n/2}$, and their finger lists need to further incorporate the points in the right half. To complete the construction for the points on the left, we first propose a simple method as follows. For each point s_i in $s_{1..n/2}$, we run *Build-Finger*(s_i) on the metric skip list for $s_{(n/2+1)..n}$. This will generate a list of new finger lists for each s_i (denote it as $\tilde{\mathcal{F}}_i$), “as if” s_i is at the location of $s_{n/2}$. This step, using Alg. 2, costs $O(n \log n \log \log n)$ work and $O(\log n \log \log n)$ span *whp*. As such, we would need to merge the two finger lists $\mathcal{F}_i$ and $\tilde{\mathcal{F}}_i$ for each point, where we only preserve a subset of points in $\tilde{\mathcal{F}}_i$.

Since both $\mathcal{F}_i$ and $\tilde{\mathcal{F}}_i$ have $O(\log n)$ finger lists *whp* and each finger list has a constant size of α , combining them will take $O(\log n)$ work *whp*, leading to:

$$W(n) = 2W(n/2) + O(n \log n \log \log n)$$

$$D(n) = D(n/2) + O(\log n \log \log n)$$

The recurrences yield $O(n \log^2 n \log \log n)$ work and $O(\log^2 n \log \log n)$ span, both *whp*. Note that in addition to a 2-way divide-and-conquer, we can increase the branching factor to k , and when combining, we consider merging $k - 1$ times, starting from the very last two subproblems to the front, each as mentioned above. Now the recurrences become:

$$W(n) = k \cdot W(n/k) + O(n \log n \log \log n)$$

$$D(n) = D(n/k) + (k - 1) \cdot O(\log n \log \log n)$$

The solutions to the recurrences imply $O(n \log n \log_k n \log \log n)$ work and $O(k \log n \log_k n \log \log n)$ span, both *whp*. By plugging in $k = n^\epsilon$ for some constant $\epsilon > 0$, we can achieve $O(n \log n \log \log n)$

work but only a polynomial span.

5.2 Overview of the Work-Efficient Algorithm

From Sec. 5.1, we can see that a straight-forward divide-and-conquer algorithm inevitably leads to redundant computation. This motivates us to strictly follow the computation in the random-walk algorithm shown in Alg. 5 to maintain work-efficiency, while still leveraging a 2-way divide-and-conquer strategy to achieve high parallelism. The key remaining question is whether in this case, the two subproblems can be *merged* in polylogarithmic span; with $O(\log n)$ recursion depth, this would imply an overall polylogarithmic span. We demonstrate that this is indeed achievable by giving an algorithm in Alg. 6, which finally leads to an $O(n \log n)$ work-efficient construction algorithm with polylogarithmic span.

THEOREM 5.1. *Given a randomly permuted point set $S = \{s_{1..n}\}$ with a constant expansion rate and a constant α , the metric skip list can be built in $O(n \log n)$ expected work and $O(\log^3 n)$ span *whp*.*

We first overview the main challenge and our high-level idea. As mentioned in Sec. 5.1, the complication in the parallel divide-and-conquer algorithm stems from the fact that the points of the first (left) half are unaware of points in the second (right) half. After both recursive calls finish, the left-half finger lists are incomplete and must further incorporate the points on the right. This necessitates a *Merge* scheme to complete the finger lists on the left. The goal is to finish the “list completion” task for all left-half points *in parallel* and *work-efficiently*. In Sec. 5.1, the *Merge* function in the naïve algorithm achieves parallelism by letting each point on the left restart a random walk from the first point on the right, i.e., $s_{n/2+1}$. As such, all random walks are independent with each other. However, this is work-inefficient since the computation differs from the sequential algorithm. During the process, more finger lists may be created and then merged/discarded, causing extra work.

Intuitively, an efficient simulation of the sequential process should let each point restart the random walk from *itself*, following the same route as the sequential algorithm. Therefore, for some s_i on the left, its starting point on the right should not simply default to $s_{n/2+1}$; rather, it should be guided by its last “hop” on the left in the original random walk. However, when this “previous hop”, which is another point on the left half, also remains uncompleted, a dependence arises, which hinders parallelism.

With this in mind, we now formally define the dependence structure among the points. Given a randomly permuted sequence of input points $S = \{s_{1..n}\}$, we consider the standard sequential construction of finger lists for all points from $s_{n'}$ to s_1 . For each point s_i , its corresponding random walk terminates at its last focus point s_{cur} (line 16 in Alg. 2). In this case, we say s_{cur} **controls** s_i (and s_{cur} is s_i 's **control point**).

Now consider $S = \{s_{1..n'}\}$ as a left-side subproblem in the divide-and-conquer scheme. In this case, we will need to complete (i.e., *Merge* the points on the right into) the finger lists for each s_i by simulating random walks from s_i . The random walk should normally advance to its control point s_{cur} , and then, it should be guided by the completed version of $\mathcal{F}_{cur}$ to advance to the right side. Clearly, the random walk of s_i cannot determine the next hop *next* before s_{cur} 's finger lists are fully ready, thereby incurring a dependence. In other words, for a point s_i , if we consider the intended random

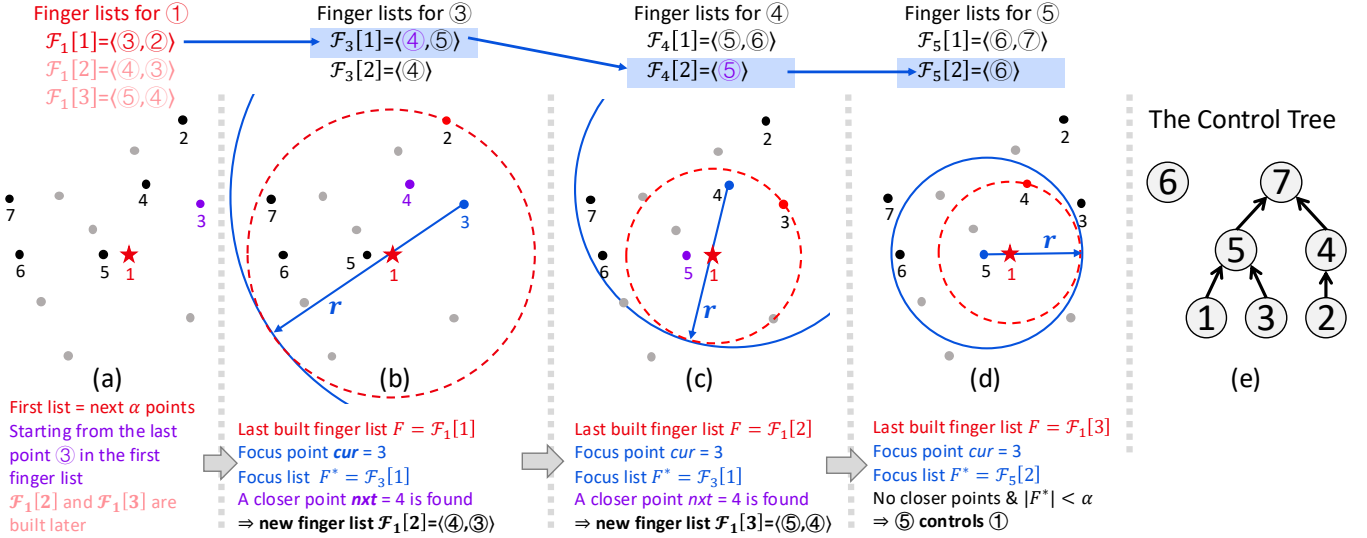


Figure 2: Illustration of parallel construction when processing point $i = 1$, and the control tree. Points 1–7 are in the current (recursive) subproblem. (a)–(d) also illustrate the sequential construction of finger lists for s_1 , showing the random walk steps from it. The point under construction ($i = 1$) and its finger lists are marked red. The focus list (F^*) of each step are in blue background. The focus point cur at each step are marked as blue dots. Red dashed circles are the ranges of the last built finger list ($F.radius$), centered with point $i = 1$. Blue solid circles are the ranges of the focus list ($F.radius$), centered with the focus point cur . The next point nxt selected are marked purple. Arrows at the top show the random walk among the finger lists. Figure (e) is the control tree.

walk process in the full list (both left and right halves considered), then its control point is exactly the last hop before advancing across the boundary between the two halves.

We present an illustration of the control point and the dependence caused by it in Fig. 2. In this example, we have points $s_{1..7}$ as the left-side subproblem in a divide-and-conquer scheme. In the subproblem on $s_{1..7}$, the finger lists of s_1 are constructed by initiating a random walk, which goes through s_3, s_4, s_5 . It terminates at $\mathcal{F}_5[2] = \langle 6 \rangle$, since no closer point is found, and this list is incomplete (line 16 in Alg. 2). By definition, s_5 controls s_1 . Since $\mathcal{F}_5[2]$ is incomplete, we can expect more points to be added with the right half considered. For s_1 , again with the right half considered, its next focus point will be determined by what other points are added to $\mathcal{F}_5[2]$: either a closer point, or, if all points are still too far, the last (largest index) will be selected. In both cases, nxt will need to wait for $\mathcal{F}_5$ to incorporate information on the right. Therefore, the Merge algorithm must first settle down s_5 before processing s_1 .

For a sequence $S = \{s_{1..n'}\}$, we define the **control tree** as a graph (forest), where each point is a vertex, and if s_i is controlled by s_j , we add an edge from i to j . For a left subproblem on S , its control tree captures the dependence of processing all the points in the Merge process. We illustrate this in Fig. 2.

A Useful Fact about Control Points. For a point s_i initially blocked by its control point s_{cur} , once s_{cur} finds its next hop on the right, the fully-built finger lists on the right will allow the random walk to proceed until the end of the right half. By definition, this process will stop at another incomplete finger list (line 16 in Alg. 2). Interestingly, this will exactly be the control point of s_i in a future recursive call, when the next time s_i (and this new control point) are both in a left subproblem. Our algorithm will use this property to avoid recomputing the control points in each recursive call.

In the remainder of this section, we present more details about the control trees and our algorithm based on it. First, we prove

that the control tree has the height of $O(\log n)$ whp, which will be detailed in Sec. 5.3. Note that the control tree is inherent to the metric skip lists and independent with any construction algorithm that generates it. With this in mind, if the tree structure is known, we can simulate random walks for the points in the tree level by level. More precisely, we can first settle all points with no control points (i.e., the roots in the control tree/forest), since they can directly advance to the right. With these points ready, we then process the next level, so on so forth. Since $|\mathcal{F}_i|$ is bounded by $O(\log n)$ whp, we can bound the span to be polylogarithmic as long as we can build the control tree efficiently.

The next question is how to find the control points efficiently, such that we can construct the control tree and execute random walks accordingly. We show this algorithm in Alg. 6 and describe it in Sec. 5.4. Given the complication of this algorithm, we will first assume the following invariant for the *advance* pointers is maintained: for all completed finger lists $\mathcal{F}_i[k]$ and $j \in \mathcal{F}_i[k]$, either $\mathcal{F}_i[k].advance[j]$ maps to the correct finger list of j , or $\mathcal{F}_i[k].advance[j]$ maps to the last completed finger list of j ; we do not maintain the *advance* pointers for the tail finger lists. We will show how the control tree can be constructed efficiently with these pointers in a work-efficient and highly parallel manner.

The final question is how to maintain the *advance* pointers. Interestingly, this appears to be the most challenging part of our algorithm. This is because in parallel, only part of the finger lists in $\mathcal{F}_i$ are built in the execution of an algorithm. This means while the next focus point can be decided, its corresponding finger list and thus the *advance* pointers are not available, unlike in the sequential case. We will show how to overcome this problem in Sec. 5.5. We primarily address two challenges. The first is how to complete the random walks work-efficiently when only partial pointers are available. The second is how to fill in the incomplete *advance* pointers work-efficiently when the target finger lists become available. To

achieve this, we again represent the dependence DAG as a tree structure, but apply the computation lazily. Combining all these components proves the main theorem, Thm. 5.1, of this paper.

5.3 Analysis of the Control Tree

As discussed, we define the concepts of the control points and thus the control tree above to capture the dependence structure in the parallel construction process. We now show that the tree height is shallow, which indicates high overall parallelism.

LEMMA 5.1. *Given a randomly shuffled sequence of points $S = \{s_1..n\}$, its corresponding control tree has the height $O(\log n)$ whp.*

Proof. Let the control point of point s_i be denoted by $s_{C(i)}$. We now claim that if $C(i) \neq i$, then with at least half probability we have $C(i) \geq (i+n)/2$. Namely, $s_{C(i)}$ is more likely to be in the second half of $s_1..n$. Note that $s_{(n-\alpha+1)..n}$ are not controlled by any point.

Let s_m be the closest point to s_i in $s_{(i+1)..n}$. Based on the random walk (see Sec. 3.3), s_m must be visited in the random walk during the construction of $\mathcal{F}_i$, so we must have $C(i) \geq m$. Since S is shuffled uniformly at random, with a half chance m is at least $(i+n)/2$, so is $C(i)$. In addition, the probability distribution is independent for the transition between i and $C(i)$ and the transition between $C(i)$ and $C(C(i))$ —this is due to the same reason as explained in the analysis of the KR algorithm for metric skip lists construction in Sec. 3.3.

With the distribution of $C(i)$, we can now bound the control tree depth easily. Consider the path starting from a node i . Every time we chase $C(i)$, it can be considered as a coin flipping with a half chance cutting the remaining range by half. These are the simple Bernoulli trials, so it takes $O(\log n)$ whp coin flips to see $\log n$ successful outcomes, which upper bounds the path lengths from i . Taking the union bound across all points proves this lemma. $\square$

In our algorithm, we use $C[i]$ to store the control point of s_i at the current recursive level. This value will be updated as the algorithm proceeds, based on the *useful fact* mentioned in Sec. 5.2.

5.4 The Construction Algorithm

Alg. 6 presents the core ideas of our parallel construction algorithm. Here, for the ease of understanding, we use $\mathcal{F}_{cur}(r)$ to acquire the focus list, and later show how to replace it with *.advance* pointers and the *align* function in Sec. 5.5.

Our *Parallel-Build* function recursively builds the finger lists for the left and right halves of the input in parallel, and then merges them. The base case is when there are fewer than α points, in which case we cannot even construct the first finger lists and can return.

During the merge phase, we first construct the control tree using the control points of the left half, which can be done using a parallel semisort [21, 33]. Then we process the control tree layer by layer in a BFS manner, starting from the roots. In Fig. 2, we process $\{s_6, s_7\}$ in the first round, then $\{s_4, s_5\}$, and finally $\{s_1, s_2, s_3\}$.

For each point i in the current layer, we resume its finger list construction from its control point $C[i]$ by the *BuildFromRight()* function, which is similar to the sequential construction algorithm in Alg. 2 and 5. This allows us to directly advance to the right half in one hop, guided by $C[i]$, then we can continue the construction as normal. Specifically, we first set F as the last complete finger list of i so far, and will append new finger lists after it. We then compute the query radius r , adjust the finger list to $\mathcal{F}_{cur}(r)$ based on the query radius r , check the condition to find the next focus point, all using

Algorithm 6: *Parallel-Build($s_{l..r}$): Builds the finger lists $s_{l..r}$ in parallel*

Maintains: $C[i]$: the control point for s_i

```

1 if  $r - l < \alpha$  then  $C[l] \leftarrow l$ ; return                                // Base case
2  $m \leftarrow (l + r)/2$ 
3 In parallel:
4   Parallel-Build( $s_{l..m}$ )
5   Parallel-Build( $s_{(m+1)..r}$ )
6 // The Merge phase: complete the finger lists on the left half
7 Construct the control tree for  $i \in [l..m]$  in parallel
8 foreach layer of the control tree do
9   ParallelForEach point  $s_i$  in this layer do
10    if  $i + \alpha \leq r$  then BuildFromRight( $i, C[i]$ )
11 Function BuildFromRight( $i, cur$ )
12   if  $i = cur$  then                                                    // i has no completed finger list
13      $\mathcal{F}_i[1].list \leftarrow \{s_{i+1}, s_{i+2}, \dots, s_{i+\alpha}\}$ ,  $cur \leftarrow i + \alpha$ 
14   while true do
15      $F \leftarrow$  the last complete finger list of  $s_i$ 
16      $r \leftarrow d(s_{cur}, s_i) + \max(F.radius, d(s_{cur}, s_i))$ 
17      $F^* \leftarrow \mathcal{F}_{cur}(r)$ 
18     if  $\exists s_j \in F^*$  s.t.  $d(s_j, s_i) < \max(F.radius, d(s_{cur}, s_i))$  then
19        $nxt \leftarrow$  the first  $j$  with this property in  $\mathcal{F}_{cur}(r)$ 
20       if  $d(s_{nxt}, s_i) < F.radius$  then
21          $F_{new} \leftarrow$  replacing the farthest point in  $F$  with  $s_{nxt}$ 
22         Append  $F_{new}$  to  $\mathcal{F}_i$ 
23     else
24       if  $|F^*| < \alpha$  then
25          $C[i] \leftarrow cur$ 
26         Build the tail and break
27       else  $nxt \leftarrow$  the last index in  $F^*$ 
28    $cur \leftarrow nxt$ 

```

the same way as the sequential algorithm. Similarly, the random walk should terminate when we reach some $F^* = \mathcal{F}_{cur}(r)$, where no closer point is found and F^* is incomplete. In this case, the next focus point is also outside (to the right) the current subproblem. Based on the useful fact mentioned in Sec. 5.2, the terminating point cur is the control point of i in a future level of recursion. Therefore, we set $C[i] = cur$ and terminate the process (line 23).

In this algorithm, building the control tree takes linear work and $O(\log n)$ span, both whp. If we use binary search to find $\mathcal{F}_{cur}(r)$, Alg. 6 directly gives us a parallel construction algorithm with $O(n \log n \log \log n)$ work whp. The span bound is $O(\log^3 n)$ whp: $O(\log n)$ levels of recursion, $O(\log n)$ levels in the control tree whp, and the random walk in each invocation of *BuildFromRight*.

5.5 Maintaining the .advance Pointers

The final piece in our parallel algorithm is to efficiently maintain the *.advance* pointers. As such, we can achieve the $O(n \log n)$ expected work bound as in our sequential algorithm. To do this, we need to address two questions: first, how to continue the random walk efficiently without all the *.advance* being finished; and second, how to efficiently fill in the *.advance* pointers once they become available. The answers, at a high level, are not complicated, and we will present them in this section. However, the algorithm with full details may be involved. Due to the space limit, we provide the full algorithm and in-depth explanation in the full version [20].

For the first question, the case happens in the tail of the fin-

ger lists of the left subproblem. When the points from the right subproblem are merged in, we need to fill in the points and set the *.advance* pointers properly. To do this, we just start from the lowest level and use the *align* function to move up and find the corresponding target. Due to the constant expansion rate, the cost of moving is asymptotically the same as the number of focus points from the right half. The solution for the second question, building the *.advance* pointers, is slightly more complicated. The *.advance* pointers are computed from three sources:

- Initialization: $\mathcal{F}_i[1].advance[j]$ points to $\mathcal{F}_j[1]$,
- Push-down: $\mathcal{F}_i[k].advance[j]$ points to $\mathcal{F}_i[k-1].advance[j]$,
- Shift focus: $\mathcal{F}_i[k].advance[j]$ points to $F.advance[j]$.

An *align* is then called to calibrate the pointers to the correct radius.

The first two cases are simple as the base cases or can be processed by moving the pointers downward when possible. The third case introduces new points into the finger lists and can change the radius, and in divide-and-conquer, it is possible that the target finger list is not ready but the random walk can continue and create this *advance* pointer. We note that all these *advance* pointers created by “shift focus” align with the random walk, so when following the sequential execution, the dependences between all these pointers form a tree structure that resembles the control tree and also has $O(\log n)$ depth *whp*. We refer to this as the *advance tree*. Hence, we can use the same approach to notify the updates of the *.advance* pointers, just like notifying the points that their random walks are now available. We provide more details in the full version [20] on how the advance-tree is built and maintained.

6 Applications

Nearest neighbor search is widely used in real-world applications. We can use our algorithm for the metric skip lists as the subroutine for these applications and achieve new parallel solutions. For space limit, we discuss these applications in the full version of this paper [20], including bichromatic closest pair (BCP) [3], density-based clustering [24, 58], and k -NN graph construction.

7 Related Work

NNS Algorithms. Nearest-neighbor search has been studied extensively in both Euclidean and general metric settings. Classical exact data structures include k -d trees [7, 49] and other spatial trees [9, 25, 29, 39, 48], navigating nets [42], cover trees [8, 32], metric skip lists [40], and others [14, 50]. Finding or even approximating nearest neighbor in the worst case requires linear time [1]; hence, known sublinear bounds are all under certain assumptions. A complementary line of work focuses on approximate search, including locality-sensitive hashing, randomized space partitions, graph-based methods, and additional techniques such as quantization [35, 43, 47, 52]. Their primary goal is on efficient searches on high-dimensional vector data, while the goals differ from the exact NNS discussed in this paper.

The most relevant prior work is the cover tree, which also addresses theoretically efficient NNS in metric spaces. A comparison is presented in Tab. 1, and here we provide more details. The cover tree was introduced by Beygelzimer et al. [8] in 2006 with $O(n \log n)$ construction work and $O(\log n)$ query time under low expansion rate. Subsequent studies [19, 22] identified flaws in the original path

compression technique, showing that the stated bounds required a bounded aspect ratio. Gu et al. [32] in 2022 proposed the work-efficient parallel version of cover tree with $O(\log^3 n \log \log n)$ span, which remains subject to both low expansion rate and bounded aspect ratio assumptions. Elkin and Kurlin [23] in 2023 proposed a new sequential cover tree variant with the stated bounds without assuming bounded aspect ratio. To our knowledge, no parallel version of that variant exists, and the techniques by Gu et al. [32] do not readily generalize to it. In contrast, our parallel algorithms for the metric skip list require only the low expansion rate assumption while achieving $O(n \log n)$ construction work, $O(\log^3 n)$ construction span, and $O(\log n)$ query cost.

Regarding practical implementations, sequential [8] and parallel [37] versions of cover trees exist; however, these implementations often relax the *separation property*, sacrificing the original theoretical guarantees. While there are no known implementations of metric skip lists, applying our algorithmic insights to practical settings remains an interesting direction for future research.

Applications. Nearest-neighbor search is a fundamental primitive in computational geometry, supporting closest-pair problems [17, 57], proximity graphs [36], and clustering [53]. In machine learning and data mining, it underlies nonparametric methods [4, 18], similarity search [35], anomaly detection [28, 51], and various clustering techniques such as DBSCAN [24, 58] and spectral clustering [26, 41, 44, 46, 59, 62]. Furthermore, it is critical in vision, robotics, and database applications, including feature matching [27], motion planning [61], and similarity joins [60]. Since these tasks rely heavily on NNS or k -NNS, faster metric structures directly accelerate these downstream algorithms. Accordingly, this paper applies our results to obtain parallel bounds for bichromatic closest pair, density-based clustering, and k -NN graph construction under the constant expansion-rate assumption [40].

8 Conclusion

This paper presents a parallel algorithm for constructing metric skip-lists in $O(n \log n)$ expected work and $O(\log^3 n)$ span *whp*. To achieve this, we first show a sequential solution with $O(n \log n)$ expected work. The main challenge in parallelizing this algorithm lies in the inherent sequential construction and the complicated dependence structure that is hard to analyze. We overcome this by adding artificial synchronization barriers that convert the dependence structure to trees with bounded heights. Putting all pieces together, we answered the decades-long open problem of how to achieve efficient nearest neighbor search on metric space with the only assumption of bounded expansion rate.

An interesting future work is to support updates in parallel or concurrent settings. While the original paper [40, 55] includes sequential updatable solutions, extending them in the parallel setting would need careful design and analysis of the dependence structure, especially for the *advance* pointers. Another future direction is to support functional updates, which is needed in some applications such as the Euclidean minimum spanning tree (EMST) and the single-linkage clustering [32].

References

- [1] Mohammad Reza Abbasifard, Bijan Ghahremani, and Hassan Naderi. 2014. A survey on nearest neighbor search methods. *International Journal of Computer Applications* 95, 25 (2014).
- [2] Ahmed Abdelkader and David M Mount. 2021. Approximate nearest-neighbor search for line segments. *arXiv preprint arXiv:2103.16071* (2021).
- [3] Pankaj K Agarwal, Herbert Edelsbrunner, Otfried Schwarzkopf, and Emo Welzl. 1990. Euclidean minimum spanning trees and bichromatic closest pairs. In *Proceedings of the sixth annual symposium on Computational geometry*. 203–210.
- [4] Naomi S Altman. 1992. An introduction to kernel and nearest-neighbor nonparametric regression. *The American Statistician* 46, 3 (1992), 175–185.
- [5] Nimar S Arora, Robert D Blumofe, and C Greg Plaxton. 2001. Thread scheduling for multiprogrammed multiprocessors. *Theory of Computing Systems (TOCS)* 34, 2 (2001), 115–144.
- [6] Jordan Bell. [n.d.]. The uniform metric on product spaces. *Lecture Notes, University of Toronto* [n.d.].
- [7] Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Commun. ACM* 18, 9 (1975), 509–517.
- [8] Alina Beygelzimer, Sham Kakade, and John Langford. 2006. Cover trees for nearest neighbor. In *International Conference on Machine Learning (ICML)*. 97–104.
- [9] Guy E Blelloch and Magdalen Dobson. 2022. Parallel Nearest Neighbors in Low Dimensions with Batch Updates. In *Algorithm Engineering and Experiments (ALENEX)*. SIAM, 195–208.
- [10] Guy E. Blelloch, Jeremy T. Fineman, Yan Gu, and Yihan Sun. 2020. Optimal parallel algorithms in the binary-forking model. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 89–102.
- [11] Robert D. Blumofe and Charles E. Leiserson. 1998. Space-Efficient Scheduling of Multithreaded Computations. *SIAM J. on Computing* 27, 1 (1998).
- [12] Georgios Chatzigeorgakidis, Sophia Karagiorgou, Spiros Athanasiou, and Spiros Skiadopoulos. 2018. FML-kNN: scalable machine learning on Big Data using k-nearest neighbor joins. *Journal of Big Data* 5, 1 (2018), 4.
- [13] Chih-Yi Chiu, Amorntip Prayoonwong, and Yin-Chih Liao. 2019. Learning to index for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 42, 8 (2019), 1942–1956.
- [14] Kenneth L Clarkson. 1997. Nearest neighbor queries in metric spaces. In *ACM Symposium on Theory of Computing (STOC)*. 609–617.
- [15] Kenneth L Clarkson et al. 2006. Nearest-neighbor searching and metric space dimensions. *Nearest-neighbor methods for learning and vision: theory and practice* (2006), 15–59.
- [16] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms (3rd edition)*. MIT Press.
- [17] Antonio Corral, Yannis Manolopoulos, Yannis Theodoridis, and Michael Vasilakopoulos. 2000. Closest pair queries in spatial databases. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 29, 2 (2000), 189–200.
- [18] Thomas Cover and Peter Hart. 1967. Nearest neighbor pattern classification. *IEEE transactions on information theory* 13, 1 (1967), 21–27.
- [19] Ryan Ross Curtin. 2015. *Improving dual-tree algorithms*. Ph.D. Dissertation. Georgia Institute of Technology.
- [20] Xiangyun Ding, Rohin Garg, Yan Gu, and Yihan Sun. 2026. Parallel Metric Skip Lists and Nearest Neighbor Search. *arXiv preprint arXiv:2606.03129*.
- [21] Xiaojun Dong, Yunshu Wu, Zhongqi Wang, Laxman Dhulipala, Yan Gu, and Yihan Sun. 2023. High-Performance and Flexible Parallel Algorithms for Semisort and Related Problems. *arXiv preprint:2304.10078* (2023).
- [22] Yuriy Elkin and Vitaliy Kurlin. 2021. A new compressed cover tree guarantees a near linear parameterized complexity for all k-nearest neighbors search in metric spaces. *arXiv preprint:2111.15478* (2021).
- [23] Yuriy Elkin and Vitaliy Kurlin. 2023. A new near-linear time algorithm for k-nearest neighbor search using a compressed cover tree. In *International Conference on Machine Learning*. PMLR, 9267–9311.
- [24] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. 1996. A Density-based Algorithm for Discovering Clusters: A Density-based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. In *KDD*. 226–231.
- [25] Raphael A Finkel and Jon Louis Bentley. 1974. Quad trees: a data structure for retrieval on composite keys. *Acta informatica* 4 (1974), 1–9.
- [26] Pasi Franti, Olli Virtajoki, and Ville Hautamaki. 2006. Fast Agglomerative Clustering Using a k-Nearest Neighbor Graph. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 28, 11 (2006), 1875–1881.
- [27] Vincent Garcia, Eric Debreuve, Frank Nielsen, and Michel Barlaud. 2010. K-nearest neighbor search: Fast GPU-based implementations and application to high-dimensional feature matching. In *IEEE International Conference on Image Processing*. IEEE, 3757–3760.
- [28] Xiaoyi Gu, Leman Akoglu, and Alessandro Rinaldo. 2019. Statistical analysis of nearest neighbor methods for anomaly detection. *Advances in Neural Information Processing Systems* 32 (2019).
- [29] Yan Gu, Yong He, Kayvon Fatahalian, and Guy Blelloch. 2013. Efficient BVH construction via approximate agglomerative clustering. In *High-Performance Graphics (HPG)*.
- [30] Yan Gu, Ziyang Men, Zheqi Shen, Yihan Sun, and Zijin Wan. 2023. Parallel Longest Increasing Subsequence and van Emde Boas Trees. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*.
- [31] Yan Gu, Zachary Napier, and Yihan Sun. 2022. Analysis of Work-Stealing and Parallel Cache Complexity. In *SIAM Symposium on Algorithmic Principles of Computer Systems (APOCS)*. SIAM, 46–60.
- [32] Yan Gu, Zachary Napier, Yihan Sun, and Letong Wang. 2022. Parallel Cover Trees and their Applications. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 259–272.
- [33] Yan Gu, Julian Shun, Yihan Sun, and Guy E. Blelloch. 2015. A Top-Down Parallel Semisort. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 24–34.
- [34] Sarel Har-Peled and Sepideh Mahabadi. 2017. Proximity in the age of distraction: Robust approximate nearest neighbor search. In *ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 1–15.
- [35] Piotr Indyk and Rameez Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *ACM Symposium on Theory of Computing (STOC)*. 604–613.
- [36] Masajiro Iwasaki and Daisuke Miyazaki. 2018. Optimization of indexing based on k-nearest neighbor graph for proximity search in high-dimensional data. *arXiv preprint arXiv:1810.07355* (2018).
- [37] Mike Izbicki and Christian Shelton. 2015. Faster cover trees. In *International Conference on Machine Learning (ICML)*. PMLR, 1162–1170.
- [38] Paul Jaccard. 1901. Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bull Soc Vaudoise Sci Nat* 37 (1901), 547–579.
- [39] Ibrahim Kamel and Christos Faloutsos. 1992. Parallel R-trees. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 21, 2 (1992), 195–204.
- [40] David R Karger and Matthias Ruhl. 2002. Finding nearest neighbors in growth-restricted metrics. In *Proceedings of the thirty-fourth annual ACM symposium on Theory of computing*. 741–750.
- [41] George Karypis, Eui-Hong Han, and Vipin Kumar. 1999. Chameleon: Hierarchical clustering using dynamic modeling. *Computer* 32, 8 (1999), 68–75.
- [42] Robert Krauthgamer and James R Lee. 2004. Navigating nets: simple algorithms for proximity search. In *ACM-SIAM Symposium on Discrete Algorithms (SODA)*. Citeseer, 798–807.
- [43] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2019. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2019), 1475–1488.
- [44] Małgorzata Łucińska and Sławomir T. Wierżchoń. 2012. Spectral Clustering Based on k-Nearest Neighbor Graph. In *Computer Information Systems and Industrial Management*. 254–265.
- [45] Prasanta Chandra Mahalanobis. 1936. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India* 2, 1 (1936), 49–55.
- [46] Markus Maier, Matthias Hein, and Ulrike Von Luxburg. 2009. Optimal construction of k-nearest-neighbor graphs for identifying noisy clusters. *Theoretical Computer Science* 410, 19 (2009), 1749–1764.
- [47] Magdalen Dobson Manohar, Zheqi Shen, Guy Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2024. Parlayann: Scalable and deterministic parallel graph-based approximate nearest neighbor search algorithms. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 270–285.
- [48] Ziyang Men, Bo Huang, Yan Gu, and Yihan Sun. 2026. Parallel Dynamic Spatial Indexes. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 150–163.
- [49] Ziyang Men, Zheqi Shen, Yan Gu, and Yihan Sun. 2025. Parallel kd-tree with Batch Updates. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 3, 1 (2025), 1–26.
- [50] Frank Nielsen, Paolo Piro, and Michel Barlaud. 2009. Bregman vantage point trees for efficient nearest neighbor queries. In *IEEE International Conference on Multimedia and Expo*. IEEE, 878–881.
- [51] Guansong Pang, Kai Ming Ting, and David Albrecht. 2015. Lesinn: Detecting anomalies by identifying least similar nearest neighbours. In *IEEE international conference on data mining workshop (ICDMW)*. IEEE, 623–630.
- [52] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient approximate nearest neighbor search in multi-dimensional databases. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 1, 1 (2023), 1–27.
- [53] Shahin Pourbahrani and Mahdi Hashemzadeh. 2022. A geometric-based clustering method using natural neighbors. *Information Sciences* 610 (2022), 694–706.
- [54] Yossi Rubner, Carlo Tomasi, and Leonidas J Guibas. 1998. A metric for distributions with applications to image databases. In *Proceedings of the Sixth International Conference on Computer Vision (ICCV)*. IEEE, 59–66.
- [55] Jan Matthias Ruhl. 2003. *Efficient algorithms for new computational models*. Ph.D. Dissertation. Massachusetts Institute of Technology.
- [56] Gregory Shakhnarovich, Trevor Darrell, and Piotr Indyk. 2008. Nearest-neighbor methods in learning and vision. *IEEE Trans. Neural Networks* 19, 2 (2008), 377.
- [57] Yufei Tao, Ke Yi, Cheng Sheng, and Panos Kalnis. 2010. Efficient and accu-

- rate nearest neighbor and closest pair search in high-dimensional space. *ACM Transactions on Database Systems (TODS)* 35, 3 (2010), 1–46.
- [58] Yiqiu Wang, Yan Gu, and Julian Shun. 2020. Theoretically-Efficient and Practical Parallel DBSCAN. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 2555–2571.
- [59] Yiqiu Wang, Shangdi Yu, Yan Gu, and Julian Shun. 2021. Fast parallel algorithms for euclidean minimum spanning tree and hierarchical spatial clustering. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1982–1995.
- [60] Chuan Xiao, Wei Wang, Xuemin Lin, Jeffrey Xu Yu, and Guoren Wang. 2011. Efficient similarity joins for near-duplicate detection. *ACM Transactions on Database Systems (TODS)* 36, 3 (2011), 1–41.
- [61] Anna Yershova and Steven M LaValle. 2007. Improving motion-planning algorithms by efficient nearest-neighbor searching. *IEEE Transactions on Robotics* 23, 1 (2007), 151–157.
- [62] Shangdi Yu, Yiqiu Wang, Yan Gu, Laxman Dhulipala, and Julian Shun. 2021. ParChain: A Framework for Parallel Hierarchical Agglomerative Clustering using Nearest-Neighbor Chain. *Proceedings of the VLDB Endowment (PVLDB)* (2021).